

Amit_Lab8 - Day 5 & 6 Assignment

C Programming - Functions, Structures & Pointers

Question 1: Find the Maximum of Two Numbers

```
#include <stdio.h>

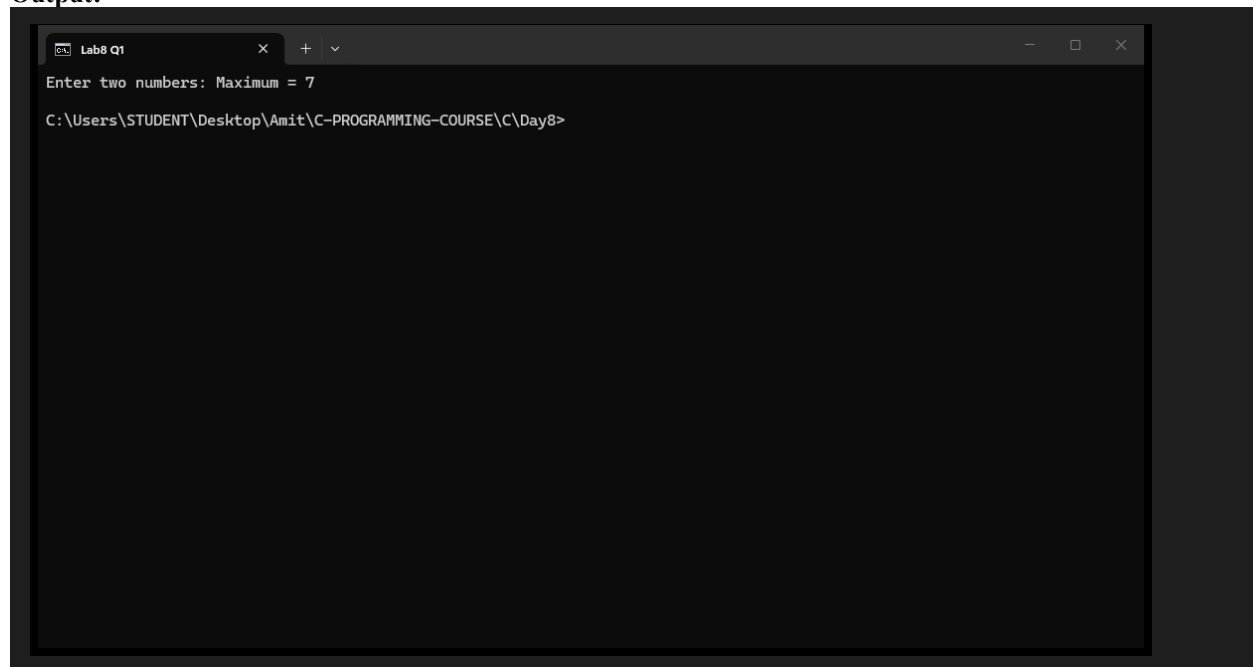
int main() {
    int a, b;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    if (a > b)
        printf("Maximum = %d\n", a);
    else
        printf("Maximum = %d\n", b);

    return 0;
}
```

Output:



```
Lab8 Q1
Enter two numbers: Maximum = 7
C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C\Day8>
```

Question 2: Check Prime Number (Write a function isPrime() that returns 1 if a number is prime, otherwise 0)

```
#include <stdio.h>

int isPrime(int n);

int main() {
    int num;

    printf("Enter a number: ");
    scanf("%d", &num);

    if (isPrime(num))
```

```

        printf("%d is a Prime Number\n", num);
    else
        printf("%d is not a Prime Number\n", num);

    return 0;
}

int isPrime(int n) {
    int i;

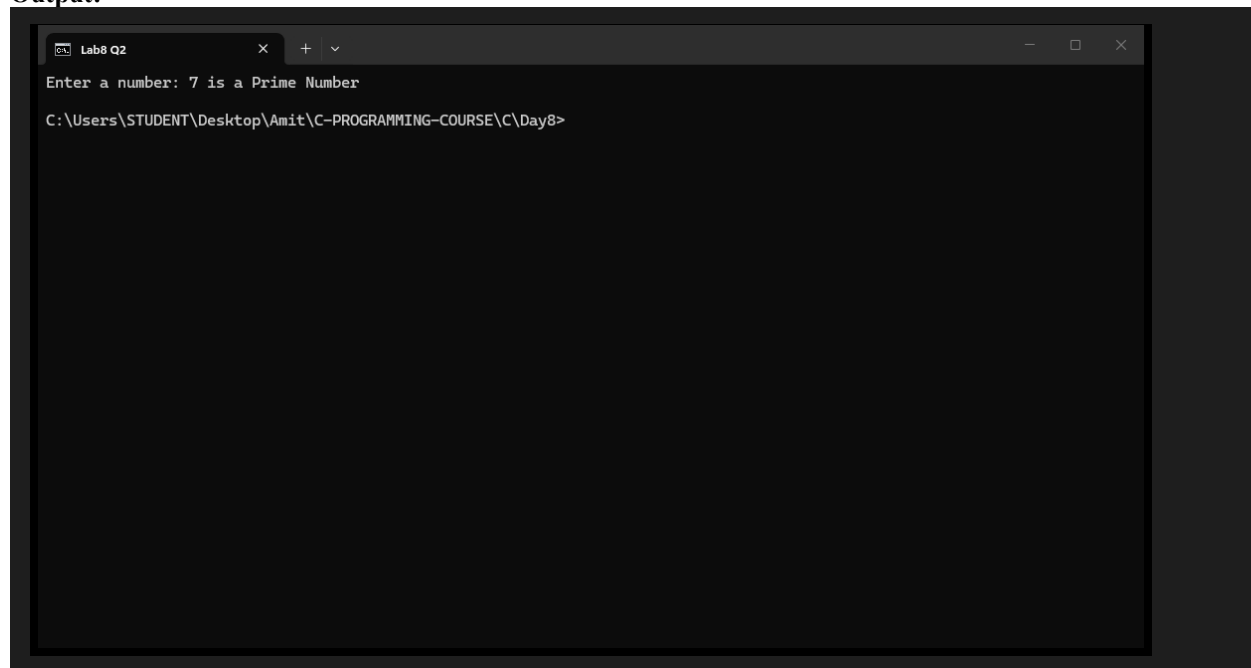
    if (n <= 1)
        return 0;

    for (i = 2; i * i <= n; i++) {
        if (n % i == 0)
            return 0;
    }

    return 1;
}

```

Output:



```

Lab8 Q2
Enter a number: 7 is a Prime Number
C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C\Day8>

```

Question 3: Reverse a Number - Write a function that returns the reverse of a given integer

```

#include <stdio.h>

int reverseNum(int n);

int main() {
    int num;

    printf("Enter a number: ");
    scanf("%d", &num);

    printf("Reversed = %d\n", reverseNum(num));

    return 0;
}

```

```

}

int reverseNum(int n) {
    int rev = 0;

    while (n != 0) {
        rev = rev * 10 + n % 10;
        n = n / 10;
    }

    return rev;
}

```

Output:

```

Lab8 Q3
Enter a number: Reversed = 321
C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C\Day8>

```

Question 4: Calculate the Factorial of a number Using Function

```

#include <stdio.h>

long factorial(int n);

int main() {
    int num;

    printf("Enter a number: ");
    scanf("%d", &num);

    printf("Factorial = %ld\n", factorial(num));

    return 0;
}

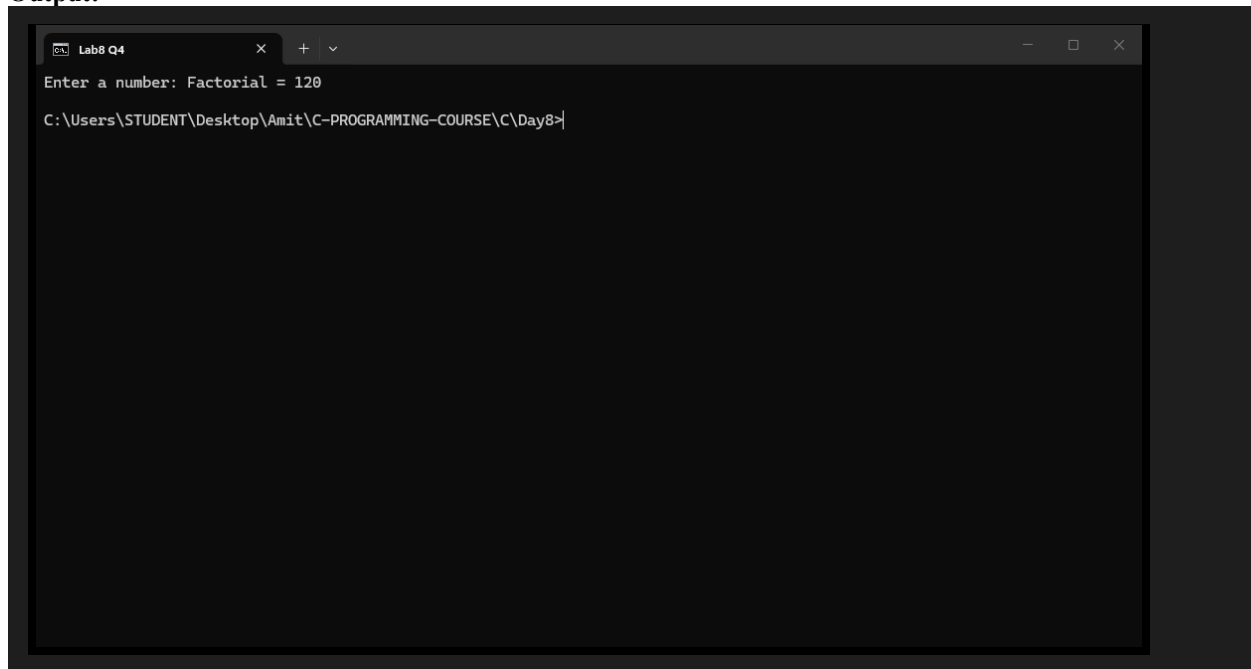
long factorial(int n) {
    int i;
    long f = 1;

    for (i = 1; i <= n; i++)
        f = f * i;
}

```

```
    return f;  
}
```

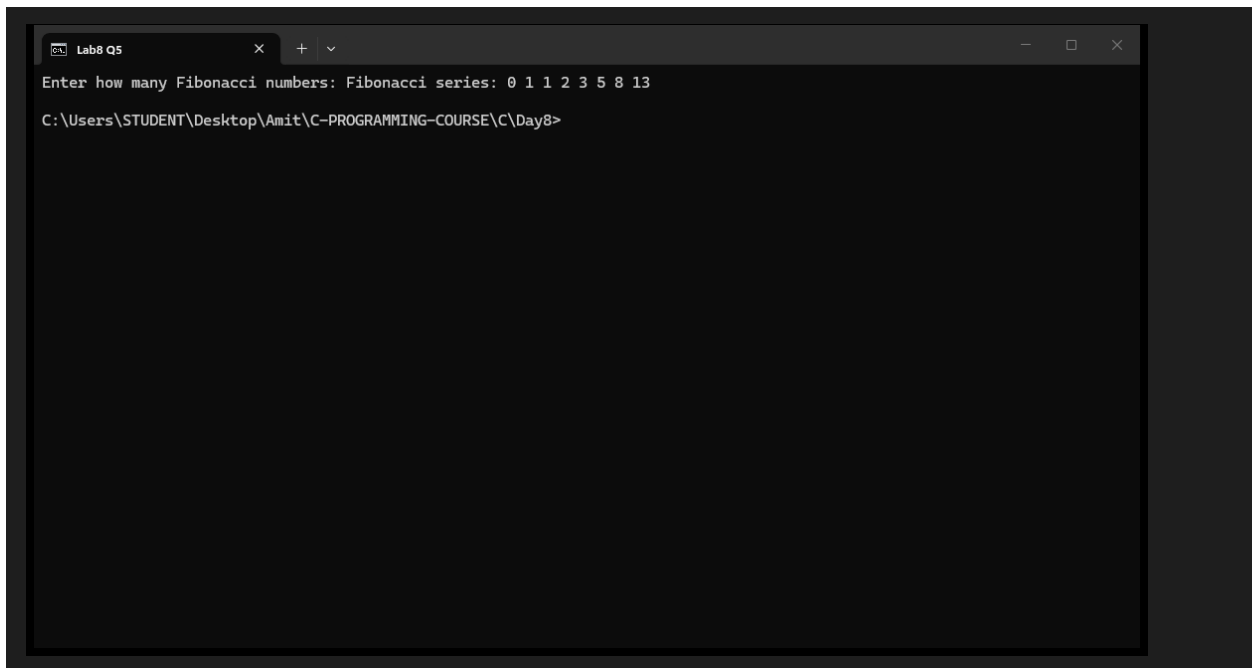
Output:



Question 5: Print the first n Fibonacci numbers

```
#include <stdio.h>  
  
int main() {  
    int n, i, a = 0, b = 1, c;  
  
    printf("Enter how many Fibonacci numbers: ");  
    scanf("%d", &n);  
  
    printf("Fibonacci series: ");  
  
    for (i = 0; i < n; i++) {  
        printf("%d ", a);  
        c = a + b;  
        a = b;  
        b = c;  
    }  
  
    printf("\n");  
  
    return 0;  
}
```

Output:



Question 6: Swap Two Numbers using call by value and call by reference, compare outputs

```
#include <stdio.h>

void swapByValue(int a, int b);
void swapByReference(int *a, int *b);

int main() {
    int x, y;

    printf("Enter two numbers: ");
    scanf("%d %d", &x, &y);

    printf("\nOriginal: x = %d, y = %d\n", x, y);

    swapByValue(x, y);
    printf("After swapByValue: x = %d, y = %d (no change)\n", x, y);

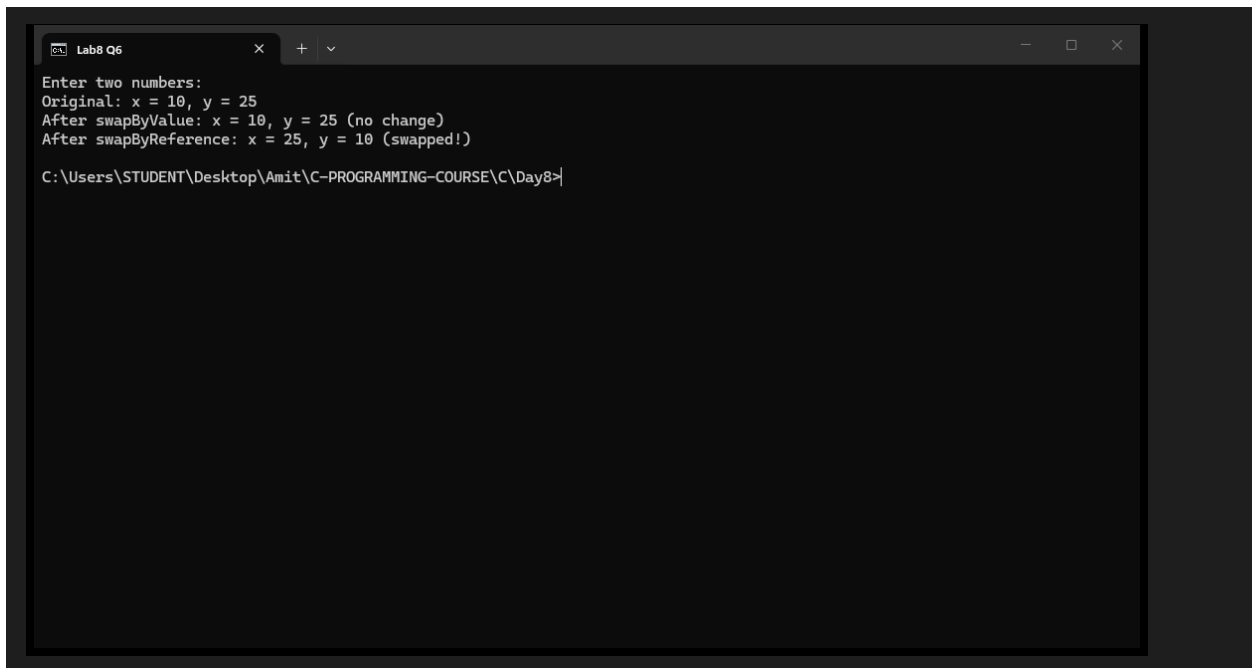
    swapByReference(&x, &y);
    printf("After swapByReference: x = %d, y = %d (swapped!)\n", x, y);

    return 0;
}

void swapByValue(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}

void swapByReference(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

Output:



```
Lab8 Q6
Enter two numbers:
Original: x = 10, y = 25
After swapByValue: x = 10, y = 25 (no change)
After swapByReference: x = 25, y = 10 (swapped!)
C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C\Day8>
```

Question 7: struct Student with roll_no, name, marks[5] - find student with highest total marks

```
#include <stdio.h>

struct Student {
    int roll_no;
    char name[50];
    float marks[5];
};

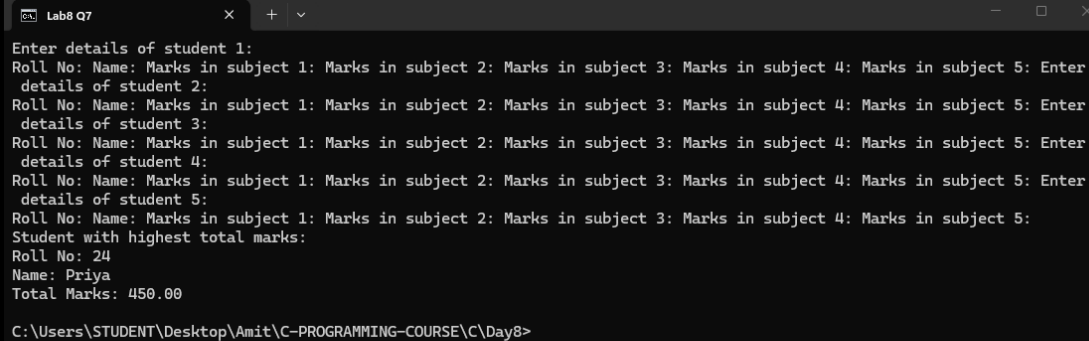
int main() {
    struct Student s[5];
    int i, j, topper = 0;
    float total, maxTotal = 0;

    for (i = 0; i < 5; i++) {
        printf("Enter details of student %d:\n", i + 1);
        printf("Roll No: ");
        scanf("%d", &s[i].roll_no);
        printf("Name: ");
        scanf("%s", s[i].name);
        total = 0;
        for (j = 0; j < 5; j++) {
            printf("Marks in subject %d: ", j + 1);
            scanf("%f", &s[i].marks[j]);
            total = total + s[i].marks[j];
        }
        if (total > maxTotal) {
            maxTotal = total;
            topper = i;
        }
    }

    printf("\nStudent with highest total marks:\n");
    printf("Roll No: %d\n", s[topper].roll_no);
    printf("Name: %s\n", s[topper].name);
    printf("Total Marks: %.2f\n", maxTotal);
}
```

```
    return 0;
}
```

Output:



```
Lab8 Q7
Enter details of student 1:
Roll No: Name: Marks in subject 1: Marks in subject 2: Marks in subject 3: Marks in subject 4: Marks in subject 5: Enter
details of student 2:
Roll No: Name: Marks in subject 1: Marks in subject 2: Marks in subject 3: Marks in subject 4: Marks in subject 5: Enter
details of student 3:
Roll No: Name: Marks in subject 1: Marks in subject 2: Marks in subject 3: Marks in subject 4: Marks in subject 5: Enter
details of student 4:
Roll No: Name: Marks in subject 1: Marks in subject 2: Marks in subject 3: Marks in subject 4: Marks in subject 5: Enter
details of student 5:
Roll No: Name: Marks in subject 1: Marks in subject 2: Marks in subject 3: Marks in subject 4: Marks in subject 5:
Student with highest total marks:
Roll No: 24
Name: Priya
Total Marks: 450.00

C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C\Day8>
```

Question 8: Employee struct (ID, Name, Salary) - find employee with highest salary

```
#include <stdio.h>

struct Employee {
    int id;
    char name[50];
    float salary;
};

int main() {
    struct Employee emp[5];
    int i, highest = 0;

    for (i = 0; i < 5; i++) {
        printf("Enter details of employee %d:\n", i + 1);
        printf("ID: ");
        scanf("%d", &emp[i].id);
        printf("Name: ");
        scanf("%s", emp[i].name);
        printf("Salary: ");
        scanf("%f", &emp[i].salary);
        if (emp[i].salary > emp[highest].salary)
            highest = i;
    }

    printf("\nEmployee with highest salary:\n");
    printf("ID: %d\n", emp[highest].id);
    printf("Name: %s\n", emp[highest].name);
    printf("Salary: %.2f\n", emp[highest].salary);

    return 0;
}
```

Output:

```
Lab8 Q8
Enter details of employee 1:
ID: Name: Salary: Enter details of employee 2:
ID: Name: Salary: Enter details of employee 3:
ID: Name: Salary: Enter details of employee 4:
ID: Name: Salary: Enter details of employee 5:
ID: Name: Salary:
Employee with highest salary:
ID: 104
Name: Neha
Salary: 70000.00

C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C\Day8>
```

Question 9: Nested struct Date in Employee - print employees who joined before a given year

```
#include <stdio.h>

struct Date {
    int d, m, y;
};

struct Employee {
    int id;
    char name[50];
    struct Date joinDate;
};

int main() {
    struct Employee emp[3];
    int i, year;
    int found = 0;

    for (i = 0; i < 3; i++) {
        printf("Enter details of employee %d:\n", i + 1);
        printf("ID: ");
        scanf("%d", &emp[i].id);
        printf("Name: ");
        scanf("%s", emp[i].name);
        printf("Date of joining (dd mm yyyy): ");
        scanf("%d %d %d", &emp[i].joinDate.d, &emp[i].joinDate.m, &emp[i].joinDate.y);
    }

    printf("\nEnter year to check: ");
    scanf("%d", &year);

    printf("\nEmployees who joined before %d:\n", year);
    for (i = 0; i < 3; i++) {
        if (emp[i].joinDate.y < year) {
            printf("ID: %d, Name: %s, Joined: %d/%d/%d\n",
                emp[i].id, emp[i].name,
                emp[i].joinDate.d, emp[i].joinDate.m, emp[i].joinDate.y);
        }
    }
}
```



```

        found = 1;
    }
}

if (!found)
    printf("No employees found.\n");

return 0;
}

```

Output:

```

C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C\Day8>
Enter details of employee 1:
ID: Name: Date of joining (dd mm yyyy): Enter details of employee 2:
ID: Name: Date of joining (dd mm yyyy): Enter details of employee 3:
ID: Name: Date of joining (dd mm yyyy):
Enter year to check:
Employees who joined before 2020:
ID: 102, Name: Priya, Joined: 20/8/2019

C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C\Day8>

```

Question 10: Library management with array of structs - add/search/display functions

```

#include <stdio.h>
#include <string.h>

struct Book {
    char title[50];
    char author[50];
    int issued;
};

void addBook(struct Book *b);
void displayBooks(struct Book *b, int count);
void searchBook(struct Book *b, int count, char *title);

int main() {
    struct Book books[20];
    int count = 0;
    int choice;
    char search[50];

    do {
        printf("\n---LIBRARY MANAGEMENT---\n");
        printf("1. Add Book\n");
        printf("2. Display Books\n");
        printf("3. Search Book\n");
        printf("4. Exit\n");
        printf("Enter choice: ");
    }
}

```

```

        scanf("%d", &choice);

        switch (choice) {
        case 1:
            addBook(&books[count]);
            count++;
            break;
        case 2:
            displayBooks(books, count);
            break;
        case 3:
            printf("Enter title to search: ");
            scanf("%s", search);
            searchBook(books, count, search);
            break;
        case 4:
            printf("Exiting...\n");
            break;
        default:
            printf("Invalid choice!\n");
        }
    } while (choice != 4);

    return 0;
}

void addBook(struct Book *b) {
    printf("Enter title: ");
    scanf("%s", b->title);
    printf("Enter author: ");
    scanf("%s", b->author);
    b->issued = 0;
    printf("Book added.\n");
}

void displayBooks(struct Book *b, int count) {
    int i;

    if (count == 0) {
        printf("No books found.\n");
        return;
    }

    for (i = 0; i < count; i++) {
        printf("Title: %s, Author: %s, Issued: %s\n",
            b[i].title, b[i].author, b[i].issued ? "Yes" : "No");
    }
}

void searchBook(struct Book *b, int count, char *title) {
    int i;

    for (i = 0; i < count; i++) {
        if (strcmp(b[i].title, title) == 0) {
            printf("Book Found!\n");
            printf("Title: %s, Author: %s\n", b[i].title, b[i].author);
            return;
        }
    }

    printf("Book not found.\n");
}

```

Output:

```
Lab8 Q10
---LIBRARY MANAGEMENT---
1. Add Book
2. Display Books
3. Search Book
4. Exit
Enter choice: Enter title: Enter author: Book added.

---LIBRARY MANAGEMENT---
1. Add Book
2. Display Books
3. Search Book
4. Exit
Enter choice: Title: C_Programming, Author: Dennis, Issued: No
Title: Data_Structures, Author: Horowitz, Issued: No

---LIBRARY MANAGEMENT---
1. Add Book
2. Display Books
3. Search Book
4. Exit
Enter choice: Enter title to search: Book not found.

---LIBRARY MANAGEMENT---
1. Add Book
2. Display Books
3. Search Book
4. Exit
Enter choice: Exiting...

C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C\Day8>
```

Question 11: Pointer basics - declare variable, print address, value, and access via pointer

```
#include <stdio.h>

int main() {
    int num = 10;
    int *ptr;

    ptr = &num;

    printf("Value of num: %d\n", num);
    printf("Address of num (&num): %p\n", &num);
    printf("Value stored in ptr: %p\n", ptr);
    printf("Value via pointer (*ptr): %d\n", *ptr);

    return 0;
}
```

Output:

```
Lab8 Q11
Value of num: 10
Address of num (&num): 0061FF18
Value stored in ptr: 0061FF18
Value via pointer (*ptr): 10

C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C\Day8>
```

Question 12: Use pointer arithmetic to traverse and print all elements of an array

```
#include <stdio.h>

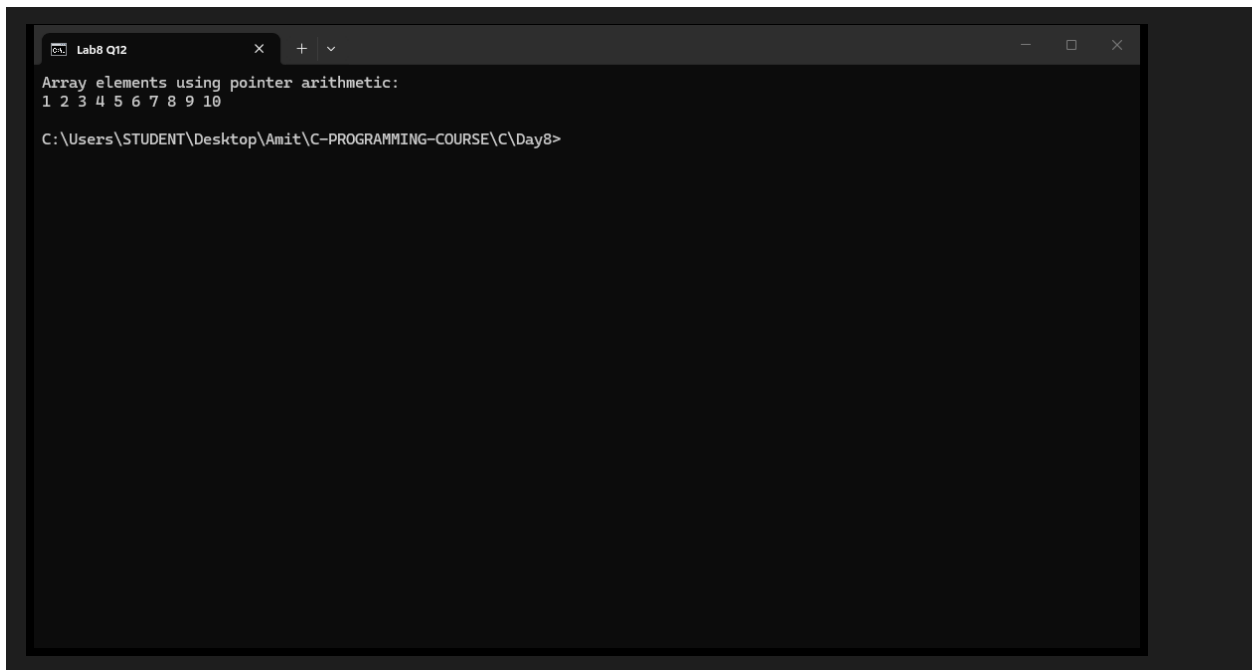
int main() {
    int arr[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    int *ptr;
    int i;

    ptr = arr;

    printf("Array elements using pointer arithmetic:\n");
    for (i = 0; i < 10; i++) {
        printf("%d ", *(ptr + i));
    }
    printf("\n");

    return 0;
}
```

Output:



Question 13: Swap Two Numbers Using Pointers

```
#include <stdio.h>

void swap(int *a, int *b);

int main() {
    int x, y;

    printf("Enter two numbers: ");
    scanf("%d %d", &x, &y);

    printf("Before swap: x = %d, y = %d\n", x, y);

    swap(&x, &y);

    printf("After swap:  x = %d, y = %d\n", x, y);

    return 0;
}

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

Output:

```
Lab8 Q13
Enter two numbers: Before swap: x = 10, y = 25
After swap:  x = 25, y = 10
C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C\Day8>
```

Question 14: Find the Largest Element in an Array Using Pointers

```
#include <stdio.h>

int main() {
    int arr[5] = {23, 45, 67, 12, 89};
    int *ptr;
    int i, max;

    ptr = arr;
    max = *ptr;

    for (i = 1; i < 5; i++) {
        ptr++;
        if (*ptr > max)
            max = *ptr;
    }

    printf("Array: ");
    for (i = 0; i < 5; i++)
        printf("%d ", arr[i]);

    printf("\nLargest element = %d\n", max);

    return 0;
}
```

Output:

```
Lab8 Q14
Array: 23 45 67 12 89
Largest element = 89

C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C\Day8>
```

Question 15: Sum of Array Elements Using Pointer Arithmetic

```
#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *ptr;
    int i;
    int sum = 0;
    float average;

    ptr = arr;

    for (i = 0; i < 5; i++) {
        sum = sum + *ptr;
        ptr++;
    }

    average = (float)sum / 5;

    printf("Array elements: ");
    ptr = arr;
    for (i = 0; i < 5; i++) {
        printf("%d ", *ptr);
        ptr++;
    }

    printf("\nSum = %d\n", sum);
    printf("Average = %.2f\n", average);

    return 0;
}
```

Output:

```
Lab8 Q15
Array elements: 10 20 30 40 50
Sum = 150
Average = 30.00
C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C\Day8>
```

Question 16: Reverse an Array Using Pointers

```
#include <stdio.h>

int main() {
    int arr[7] = {1, 2, 3, 4, 5, 6, 7};
    int *left, *right;
    int temp;
    int i;

    printf("Original array: ");
    for (i = 0; i < 7; i++)
        printf("%d ", arr[i]);
    printf("\n");

    left = arr;
    right = arr + 6;

    while (left < right) {
        temp = *left;
        *left = *right;
        *right = temp;
        left++;
        right--;
    }

    printf("Reversed array: ");
    for (i = 0; i < 7; i++)
        printf("%d ", arr[i]);
    printf("\n");

    return 0;
}
```

Output:


```
Lab8 Q16
Original array: 1 2 3 4 5 6 7
Reversed array: 7 6 5 4 3 2 1

C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C\Day8>
```

Question 17: Declare int, pointer, double pointer - display all values and addresses

```
#include <stdio.h>

int main() {
    int num = 42;
    int *ptr = &num;
    int **pptr = &ptr;

    printf("Value of the variable: %d\n", num);
    printf("Address of the variable (&num): %p\n", &num);
    printf("Value stored in the pointer (ptr): %p\n", ptr);
    printf("Address of the pointer (&ptr): %p\n", &ptr);
    printf("Value stored in the double pointer (*pptr): %p\n", *pptr);
    printf("Address of the double pointer (&pptr): %p\n", &pptr);

    return 0;
}
```

Output:

```
Lab8 Q17
Value of the variable: 42
Address of the variable (&num): 0061FF1C
Value stored in the pointer (ptr): 0061FF1C
Address of the pointer (&ptr): 0061FF18
Value stored in the double pointer (*pptr): 0061FF1C
Address of the double pointer (&pptr): 0061FF14

C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C\Day8>
```

Question 18: Change the value of an integer using only a double pointer

```
#include <stdio.h>

int main() {
    int num = 10;
    int *ptr = &num;
    int **pptr = &ptr;

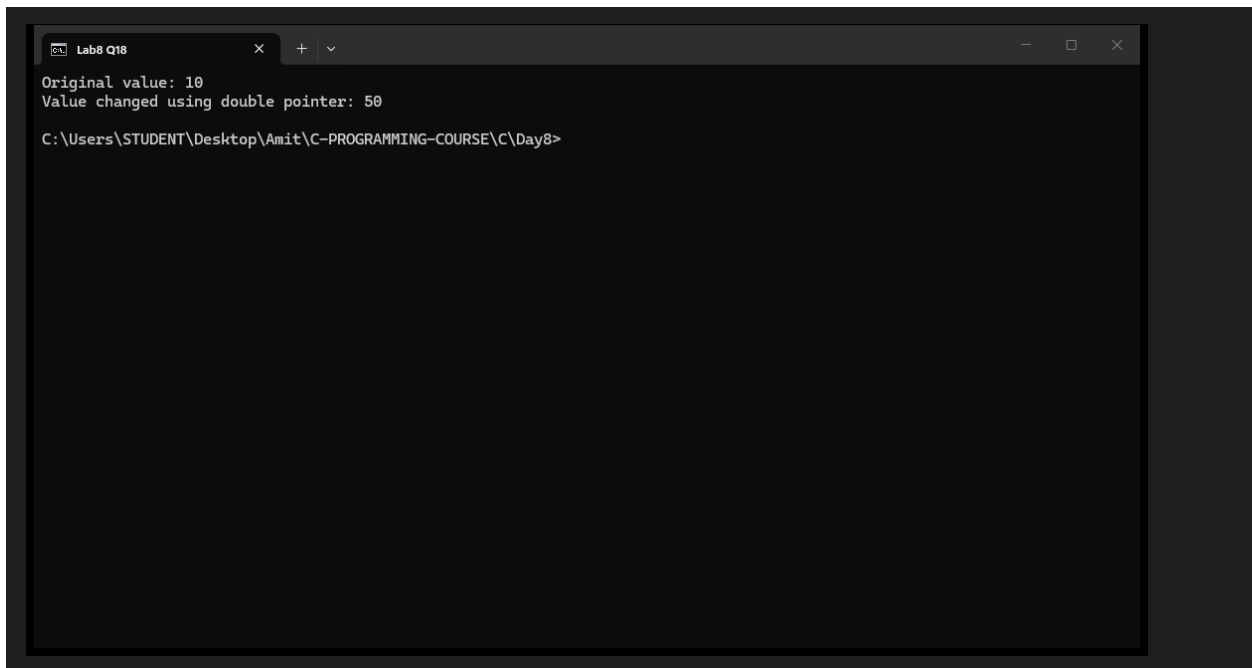
    printf("Original value: %d\n", num);

    **pptr = 50;

    printf("Value changed using double pointer: %d\n", num);

    return 0;
}
```

Output:



```
Lab8 Q18
Original value: 10
Value changed using double pointer: 50
C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C\Day8>
```

Question 19: Use pointer to iterate array and count even/odd elements

```
#include <stdio.h>

int main() {
    int arr[10] = {12, 5, 8, 7, 10, 3, 6, 9, 14, 1};
    int *ptr;
    int even = 0, odd = 0;
    int i;

    ptr = arr;

    for (i = 0; i < 10; i++) {
        if (*ptr % 2 == 0)
            even++;
        else
            odd++;
        ptr++;
    }

    printf("Array elements: ");
    ptr = arr;
    for (i = 0; i < 10; i++) {
        printf("%d ", *ptr);
        ptr++;
    }

    printf("\nEven numbers: %d\n", even);
    printf("Odd numbers: %d\n", odd);

    return 0;
}
```

Output:

```
Lab8 Q19 x + v
Array elements: 12 5 8 7 10 3 6 9 14 1
Even numbers: 5
Odd numbers: 5
C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C\Day8>
```

Question 20: Use pointer to find sum and average of all elements in integer array

```
#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *ptr;
    int i;
    int sum = 0;
    float average;

    ptr = arr;

    for (i = 0; i < 5; i++) {
        sum = sum + *ptr;
        ptr++;
    }

    average = (float)sum / 5;

    printf("Array: ");
    ptr = arr;
    for (i = 0; i < 5; i++) {
        printf("%d ", *ptr);
        ptr++;
    }

    printf("\nSum = %d\n", sum);
    printf("Average = %.2f\n", average);

    return 0;
}
```

Output:

```
Q20SumAvg
Array: 10 20 30 40 50
Sum = 150
Average = 30.00
C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C\Day8>
```

MCQ Questions: Answers

Q	Answer
1	A) 6 6
2	B) 40
3	C) 30
4	C) s.marks
5	C) static
6	C) 1 2 3
7	A) 80
8	B) 20
9	C) 10 20
10	A) 5

Explain

1. Differentiate between call by value and call by reference in C. Give suitable examples and mention when each is preferred.

In call by value, a copy of the argument is passed to the function, so changes inside do not affect the original. In call by reference, the address of the argument is passed, so the function can modify the original variable. For example swapByValue(x, y) copies x and y and swaps only the copies, while swapByReference(&x, &y) passes addresses and swaps the originals. Call by value is preferred when the function should not change the caller's data, while call by reference is needed when the function must modify caller variables or pass large structures efficiently.

2. What is recursion? Explain its advantages, disadvantages, and give one example where recursion is preferred over iteration.

Recursion is when a function calls itself to solve a problem by breaking it into smaller sub-problems. Advantages: code is simpler and more readable for problems with a recursive structure like tree traversal. Disadvantages: uses extra stack memory, slower than iteration, and may cause stack overflow for very deep recursion. Recursion is

preferred for problems like factorial, Fibonacci, Tower of Hanoi and binary tree traversal where the recursive definition naturally matches the algorithm.

3. What is a pointer? Explain the purpose of pointers in C. How do pointers improve the efficiency of programs?

A pointer is a variable that stores the memory address of another variable. Pointers are used for dynamic memory allocation, passing arrays and structures efficiently to functions, implementing data structures like linked lists and trees, and allowing functions to modify caller variables. They improve efficiency by avoiding copying large data structures, enabling direct memory access, and allowing in-place modification of data without returning copies.

4. Explain pointer arithmetic in C. Which arithmetic operations are valid on pointers, and why?

Pointer arithmetic works in units of the pointed-to type. For an int pointer p pointing to arr[0], p+1 points to arr[1] (4 bytes ahead), p+2 to arr[2], etc. Valid operations: addition/subtraction of an integer, subtraction of two pointers of the same type. Invalid operations: multiplication, division, addition of two pointers. Subtraction of two pointers gives the number of elements between them. Pointer arithmetic is safe because it respects the size of the underlying type.

5. What is a double pointer (**)? Explain its syntax, purpose, and one practical application with an example.

A double pointer is a pointer to a pointer - it stores the address of another pointer variable. Syntax: `int **pptr = &ptr;` where ptr itself is `int *`. The value is accessed as `**pptr` which dereferences twice to reach the original integer. Purpose: used for dynamic 2D arrays, modifying pointer parameters in functions, and in some data structures. For example in q18, `**pptr = 50;` changes the original integer variable through two levels of indirection.

6. Differentiate between structures and arrays in C with suitable examples.

An array stores elements of the same data type in contiguous memory, accessed by index, e.g. `int marks[5];`. A structure groups different data types under one name, accessed by member name, e.g. `struct Student{int id; float marks;};`. Arrays are homogenous (all same type), structures are heterogenous (mixed types). Arrays use indexing `marks[0]`, structures use dot operator `s.marks`. Arrays are useful for lists of the same data; structures are useful for grouping related but different fields like student records.

7. Explain the different ways of passing a structure to a function. Compare passing by value and by pointer.

A structure can be passed by value (e.g. `void display(struct Student s)`) which creates a copy of all members, or by pointer (e.g. `void update(struct Student *s)`) which passes the address. By value: safe (original not modified), but slower for large structs due to copying. By pointer: faster (only address copied), allows the function to modify the original struct, but the original can be changed unintentionally. The program in q7 demonstrates call by value where `update(s)` changes only the local copy, so the original remains unchanged.

8. Explain the four storage classes in C: auto, register, static, extern.

auto: default for local variables, stored in stack, lifetime is within the block, default value is garbage. register: stored in CPU register (hint to compiler), faster access, same scope/lifetime as auto, cannot use `&` operator. static: persists between function calls, initialized only once, default value is 0, scope is local to block. extern: declared outside any function, shares variable across files, lifetime is program duration, default value is 0. The static variable in q6 (fun) retains its value across calls, printing 1 2 3 instead of 1 1 1.

9. Explain the difference between struct and typedef struct in C. Why is typedef commonly used with structures?

Without typedef, you must write `struct Student s;` every time you declare a variable. With `typedef struct{...} Student;` you create an alias so you can write `Student s;` directly, making code shorter and cleaner. typedef is commonly used because it reduces repetition, especially when the same struct is used in many places. Both are equivalent in functionality, but typedef is preferred for readability and convenience.

10. Why is a structure assignment allowed in C, but an array assignment is not?

In C, the assignment operator (`=`) works on structures because the compiler copies all members of one struct to another in a single operation (e.g. `s2 = s1;`). Arrays are not allowed to be assigned with `=` because C treats array names as constant pointers to the first element, and you cannot assign to an array name. To copy arrays, you must use a loop or `memcpy()`. Structures support direct assignment, but arrays do not - this is a fundamental difference in how C handles these two aggregate types.